Figure 2: Benefits of continuous integration

Figure 3: Best practices of continuous integration

even if the compilation fails; or when libraries are hard coded and the path in the developer's system is different, and hence the compilation fails.

The solution

Please don't *commit* if you are not ready. This piece of advice holds good for life as well as while developing software! After each commit, build validation has to pass, all quality gates have to be cleared and things must work successfully in the pipeline.

Even if the pipeline fails, it must be fixed immediately. This failure helps the Jenkins engineer and developer to grow.

Failure should be the motive to inspire us to continue to fix problems and improve continuously even if a temporary failure creates a road block. We must not avoid build failures but address them head on. Build success is only one step away when the issues are fixed.

When you decide to go down the path of cultural transformation, think about the satisfaction you'll get when after addressing multiple failures, your software eventually gets the green status.

Benefits, best practices and challenges of CI

We can understand some of the benefits of continuous integration from Figure 2.

Best practices make it easy to implement continuous integration within an organisation, without resistance. Please refer to Figure 3.

The major challenge with continuous integration is the mindset! People resist it because all developers are so used to the IDE, its features and functionality that any new tool is not welcomed. It is important to start with the proof-of-concept and a demo of all the automation activities orchestrated in the automation server. This helps to develop a mindset that is ready to accept new practices.

Once the resistance is overcome, the development team can use CI practices effectively because it helps to boost productivity.

Continuous delivery

Continuous delivery and continuous deployment are the next logical steps when implementing DevOps practices.

Continuous integration creates an artifact that needs to be deployed in a development or QA stage, or a production environment. The next step is automated deployment in different environments. These environments can be on-premise or on the cloud. Resources can be physical or virtual machines, or containers. Services can be Infrastructure as a Service or a Platform as a Service, where the artifact can be deployed.

The continuous delivery phase represents activities to deploy an artifact in non-production environments using scripts or deployment tools. The artifact is always ready to be deployed in production. Usually, continuous delivery is a preferred practice since there's less risk involved with it.